

AERO-KB

Knowledge Base as Code cho hệ sinh thái hàng không
— từ tri thức domain đến BA Agent & Codebase-as-Knowledge

Phiên bản	0.3 — Đồng bộ tài liệu với thực trạng codebase v0.10.0
Ngày	14/07/2026
Tác giả	Lam (PM) — soạn cùng Claude
Trạng thái	Phase 1-3 đã vận hành & đóng gói phát hành · Phase 4-5 thiết kế
Đối tượng đọc	BA, Dev, SME, PM các dự án SIA

Thay đổi chính so với v0.2 (10/07/2026): engine tách khỏi domain hàng không và phát hành dưới tên **CENTER-KB** (package `center-kb` trên PyPI, CLI vẫn là `kb`); chuyển sang kiến trúc **hub-first** (v0.9.0) — hub là nguồn đọc duy nhất, `kb publish mirror` đủ L0→L3, bỏ `kb approve`; bổ sung **Web UI + REST** cùng tiến trình MCP; **hybrid search index** bền vững FTS5 + vec0 (v0.10.0); **auto-summarize** headless; **role-aware init** (hub/child, hỗ trợ Claude Code · Copilot · Cursor); **Windows first-class**; **release gate T1-T4** + Docker/PyPI. **Phase 4 — BA Agent** và **Phase 5 — Developer Package** giữ nguyên định hướng, đã hiệu chỉnh theo nền tảng mới.

1. Tóm tắt

AERO-KB là chương trình knowledge base cho hệ sinh thái hàng không, xây trên engine **CENTER-KB** — một bộ kit (Python package + CLI + MCP server + Web UI) biến các tài liệu chính thống — ARINC 424, ICAO Annex, quy định khai thác mặt đất, SOP nội bộ — thành **knowledge base sống ngay trên source code**, version bằng Git và tối ưu cho AI agent truy vấn. Từ v0.2, engine đã tách domain-agnostic và phát hành công khai: `pip install center-kb`, image Docker `ghcr.io/vuonglq01685/center-kb`; repo AERO-KB là instance vận hành với dữ liệu hàng không.

Vấn đề cốt lõi không đổi: tài liệu hàng không cực lớn (một mình ARINC 424 đã hơn 1.000 trang), trong khi agent chỉ có ngân sách context giới hạn và mỗi token đều có chi phí. AERO-KB giải quyết bằng ba cơ chế: **compile once, query many** (cô đọng một lần thành cấu trúc 4 tầng L0-L3, truy vấn về sau gần như miễn phí), **progressive disclosure** (agent chỉ load đúng phần cần), và **requirement mang theo context** (mỗi requirement đính kèm block citation máy-đọc-được, pin đúng version KB).

So với v0.2 (soạn ngày 10/07/2026, ngay khi Phase 1-3 vừa chạy PoC), v0.3 ghi nhận **~270 commit tiến hoá trong 4 ngày**, với 4 thay đổi kiến trúc đáng kể:

- **Hub-first single source (v0.9.0):** mọi truy vấn (`kb query`, MCP, Web UI) chỉ đọc `federation/` trên hub; `.kb/` cục bộ là bản nháp. `kb publish` mirror **đủ L0→L3** lên hub và mở PR — merge PR đó là cổng duyệt duy nhất đưa nội dung "live". `kb approve` và flow auto-reviewed bị loại bỏ.
- **Hybrid search index (v0.10.0):** chỉ mục bền vững SQLite — FTS5 (keyword) + vec0 KNN (semantic, embedding local) — hợp nhất bằng RRF; làm tươi tăng dần theo content-hash.
- **Một tiến trình phục vụ cả agent lẫn người:** HTTP MCP (`/mcp`) + REST (`/api`) + Web UI (`/ui`) chung một process, chung một bearer token.
- **Đóng gói sản phẩm:** role-aware `kb init` (hub/child), auto-summarize headless, Windows first-class, release gate T1-T4, phát hành PyPI + GHCR.

Trên nền đó, hai hướng phát triển tiếp theo **giữ nguyên từ v0.2**, cập nhật lại điểm tựa:

- **Phase 4 — BA Agent:** agent sinh User Story / ticket theo format chuẩn, bám sát knowledge base, kèm **citation nguồn, sequence diagram, use case** và **business flow**. Một phần nguyên liệu đã ship sớm: flow "kb_search trả mọi ứng viên → BA xác nhận → kb_context_new" đã có trong MCP.
- **Phase 5 — Developer Package:** package cài trong từng repo con — (a) kết nối KB để **làm ticket BA giao** (nửa này đã có sẵn phần lớn nhờ child-scaffold của `kb init`), và (b) cơ chế **liên tục tổng hợp & mô tả code structure, database, 3rd-party integration và các repo liên quan**, sinh bằng lệnh và publish lên hub tự động qua CI.

Mục lục

PHẦN I — HIỆN TRẠNG (Phase 1-3 đã triển khai & đóng gói)

2. Bài toán & mục tiêu	4
3. Nguyên tắc thiết kế	5
4. Kiến trúc tổng thể	6
5. Cấu trúc knowledge base 4 tầng	7

6. Pipeline ingest & vòng đời section	8
7. Retrieval: hybrid search index	9
8. Federation hub-first, remote MCP & Web UI	10
9. Interface: CLI & MCP	12
10. Flow theo từng role	14
11. Governance, versioning & bản quyền	15
12. Bảng chứng vận hành & chất lượng phát hành	16

PHẦN II — ROADMAP (Phase 4-5 thiết kế)

13. Phase 4 — BA Agent	18
14. Phase 5 — Developer Package & Codebase-as-Knowledge	20
15. Vòng khép kín & roadmap tổng hợp	22
16. Rủi ro & giảm thiểu	23
Phụ lục A — Ví dụ ticket do BA Agent sinh	24
Phụ lục B — Ví dụ code-knowledge doc (Phase 5)	25

2. Bài toán & mục tiêu

2.1 Bối cảnh

Các dự án hàng không (Talora / ARINC 424 dashboard, các hệ thống SIA...) phụ thuộc vào một khối lượng tài liệu chuẩn ngành khổng lồ: đặc tả ARINC 424, các ICAO Annex, quy định khai thác mặt đất, SOP nội bộ. Khi đưa AI agent vào quy trình — BA viết requirement qua agent, Dev code bằng agent — mọi câu trả lời đúng đều phải bám vào các tài liệu này.

Cách làm ngây thơ (đưa cả tài liệu vào context, hoặc RAG thuần trên raw text) gặp ba vấn đề: **chi phí token** bùng nổ vì mỗi truy vấn kéo theo hàng chục nghìn token dư thừa; **chất lượng không ổn định** retrieval trên raw text dễ trả nhầm section và agent "tự tin" trích sai điều khoản — rủi ro nghiêm trọng với domain hàng không; và **không có traceability** — không trả lời được "requirement này viết dựa trên tài liệu nào, bản sửa đổi nào", điều gần như bắt buộc trong môi trường có audit.

2.2 Mục tiêu & kết quả đo được

Mục tiêu	Thước đo	Hiện trạng
Giảm chi phí token cho truy vấn tri thức domain	≥ 90% so với nạp raw document; index thường trực < 1K token	ĐẠT ≥99%
Mọi requirement / câu trả lời agent đều có citation	100% requirement đạt Definition of Ready có block <code>kb-context</code>	ĐẠT
BA và Dev dùng chung một nguồn tri thức, đúng version	Citation trong ticket resolve được về đúng section, đúng commit hub	ĐẠT
Cập nhật tài liệu (amendment) là sự kiện phát hiện được	<code>kb doctor</code> cảnh báo mọi ref bị stale	ĐẠT
Hoạt động với mô hình microservices / multi-repo	Thêm 1 repo mới không làm tăng đáng kể token cost của truy vấn	ĐẠT
Dùng lại được cho domain khác, cài đặt chuẩn hoá	Engine domain-agnostic, phát hành PyPI/ Docker, gate hoá release, chạy trên Windows	ĐẠT v0.10.0
Requirement giàu ngữ cảnh (story, use case, diagram) tự sinh từ KB	BA Agent sinh ticket đủ story + AC + diagram, cited	PHASE 4
Code structure / DB / integration của repo con là tri thức tra được	Sinh tự động qua CI, publish lên hub, không bao giờ stale	PHASE 5

2.3 Ngoài phạm vi (non-goals)

- Không thay thế hệ thống quản lý tài liệu chính thống của ICAO / ARINC / hãng.
- Không phải search engine toàn văn cho người dùng cuối — đối tượng phục vụ là AI agent và quy trình BA/Dev. Web UI `/ui` (đã có từ v0.5) đúng như tuyên bố ở v0.2: chỉ là lớp UI mỏng đặt trên engine sẵn có.
- Không OAuth/SSO cho HTTP MCP — bearer token duy nhất, chạy trong mạng nội bộ/VPN.

3. Nguyên tắc thiết kế

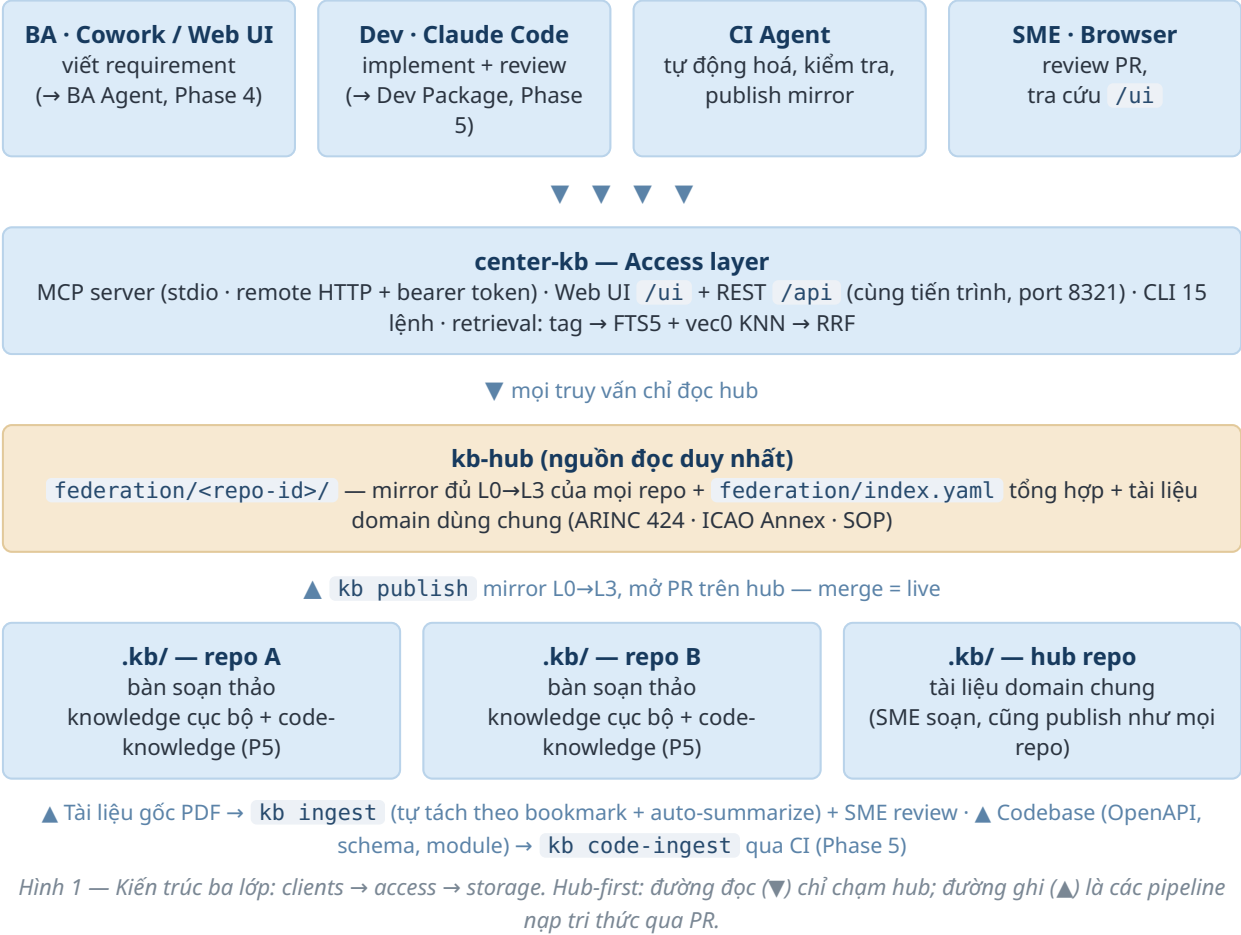
Bảy nguyên tắc dưới đây định hình toàn bộ hệ thống — và tiếp tục ràng buộc mọi thiết kế Phase 4–5. Sáu nguyên tắc đầu giữ nguyên từ v0.2; nguyên tắc thứ bảy (hub-first) được chốt ngày 13/07/2026.

Nguyên tắc	Ý nghĩa
Docs-as-code	KB là các file markdown/YAML nằm trong repo, version bằng Git, review bằng PR — không cần database hay hạ tầng thường trực.
Progressive disclosure	Tri thức chia tầng L0→L3; agent đi từ nông xuống sâu, chỉ nạp phần cần thiết (cùng pattern với SKILL.md của Anthropic).
Compile once, query many	Chi phí LLM (summarize) chỉ trả một lần lúc ingest; truy vấn về sau là đọc file thuần + routing bằng code.
Citation bắt buộc	Mọi output của agent dựa trên KB phải cite <code>doc-id §section</code> . Với hàng không, traceability không phải tùy chọn.
Human-in-the-loop	Nội dung do LLM sinh phải được người thẩm định trước khi trở thành "sự thật"; agent chỉ tạo bản nháp. Cổng thẩm định vận hành là PR trên hub (§6, §8).
Routing bằng code, không bằng LLM	Tag matching, FTS5, vec0 KNN, RRF ranking chạy deterministic ngoài model — không tốn token, không sai ngẫu nhiên.
Hub-first, một nguồn đọc duy nhất	Mọi máy truy vấn (CLI/MCP/Web) chỉ đọc <code>federation/</code> trên hub. <code>.kb/</code> cục bộ là bản soạn thảo — không ai query nó; nội dung chỉ "live" sau khi PR publish trên hub được merge.

Bảng biểu là bất khả xâm phạm. Trong toàn hệ thống, bảng biểu (mã hiệu, độ dài field, kiểu ký tự) **không bao giờ** do LLM viết lại — luôn được code trích xuất y nguyên từ nguồn, và `kb build` tự động so khớp bảng ở L2 với L3, lệch một ký tự là chặn merge. Đây là chốt an toàn quan trọng nhất.

4. Kiến trúc tổng thể

AERO-KB gồm ba lớp: **storage** (các thư mục `.kb/` nơi soạn thảo và `federation/` trên kb-hub — nguồn đọc duy nhất), **access** (CLI + MCP server + Web UI/REST), và **clients** (các agent và con người). Toàn bộ storage là file thuần trong Git — server chỉ là lớp giao tiếp, không phải nền móng.



Vai trò hub/child giờ là cấu hình chính thức. kb init hỏi (hoặc nhận --kind hub|child) và ghi kind: vào .kb/config.yaml. **Hub** giữ federation/, chạy dịch vụ MCP/Web dùng chung (scaffold docker-compose + kb docker-setup sinh .env và token). **Child** soạn tri thức cục bộ và publish lên hub; scaffold nhận .mcp.json trở HTTP hub, slash commands và skill cho Claude Code, GitHub Copilot và Cursor. Repo do SME dựng (nơi tập trung tri thức domain quan trọng nhất) chính là hub của AERO-KB.

Vì sao cần MCP server? Không có nó, KB vẫn hoạt động (agent Read/Grep trực tiếp). MCP server bổ sung ba giá trị: (1) routing chạy bằng code — không tốn token, không sai ngẫu nhiên; (2) kiểm soát token budget tập trung; (3) một interface chung cho mọi client. Dạng cục bộ là **stdio** khai báo trong .mcp.json; dạng dùng chung là **remote HTTP** có bearer token — đổi transport không đổi code tool. Từ v0.5, cùng tiến trình HTTP đó phục vụ luôn con người qua /ui.

5. Cấu trúc knowledge base 4 tầng

Mỗi tài liệu sau khi ingest được tổ chức thành 4 tầng, từ nông đến sâu. Nguyên tắc: **tầng càng nông càng rẻ và càng hay được đọc; tầng sâu chỉ nạp khi thực sự cần**. Cấu trúc này không đổi từ v0.2 — mọi tính năng mới (hub mirror, hybrid index, code-knowledge Phase 5) đều đọc/ghi đúng format này.

Tầng	Là gì	Nội dung	Kích thước
L0	<code>index.yaml</code> — danh mục kho	Mỗi tài liệu 1 dòng: id, tags, revision, mô tả ngắn. Agent luôn thấy "kho có gì".	~187 token · toàn kho
L1	<code>_manifest.yaml</code> (mỗi tài liệu)	Cây section + 1 câu summary/section + status. Load khi tài liệu được kích hoạt.	~1–30K token / tài liệu
L2	section đã cô đọng (<code>.md</code>)	Summary giữ nguyên ngôn ngữ nguồn, có cấu trúc; bảng/giá trị quan trọng giữ nguyên vẹn . Load đúng section cần.	~0,5–2K token / section
L3	trích xuất gốc (<code>.raw.md</code>)	Nguyên văn theo section — chỉ đọc khi cần verify tuyệt đối chính xác.	theo nhu cầu

Hình 2 — Bốn tầng progressive disclosure. Đa số truy vấn dừng ở L1–L2.

5.1 Cấu trúc thư mục (dữ liệu thật hiện tại)

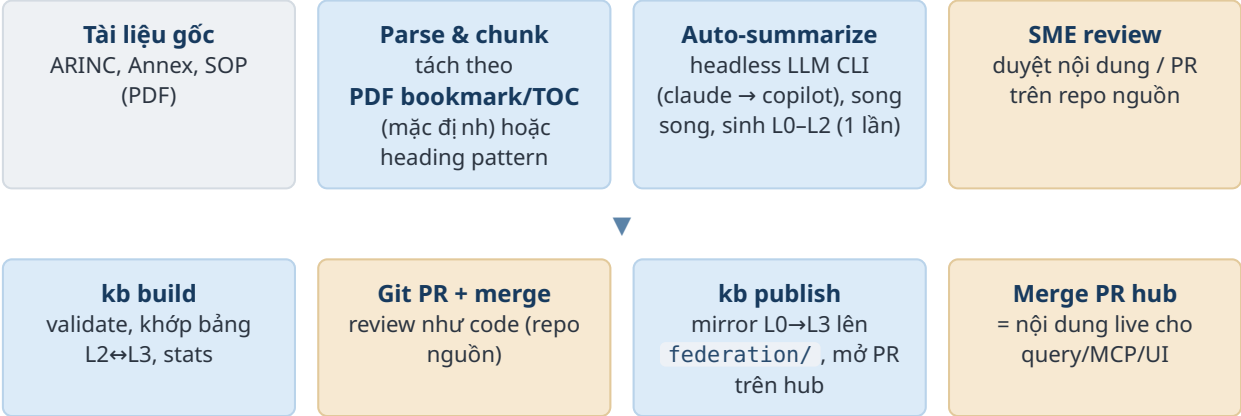
```
.kb/
├─ index.yaml                # L0 – danh mục toàn bộ KB (187 token)
├─ config.yaml              # hub: · repo_id: · kind: hub|child (từ v0.9)
├─ arinc-424/
│   ├── _manifest.yaml       # L1 – 325 section + summary
│   ├── ch5-navigation-data-...md # L2 – nội dung cô đọng
│   └─ ch5-navigation-data-...raw.md # L3 – trích xuất gốc
└─ icao-annex-3/
    ├── _manifest.yaml       # L1 – 4 section (Ch.2)
    ├── ch2-general-provisions.md # L2
    └─ ch2-general-provisions.raw.md # L3
```

5.2 Ví dụ một entry trong index.yaml (L0)

```
docs:
- id: arinc-424
  title: "ARINC 424 – Navigation System Data Base"
  revision: "Supplement 22"
  tags: [arinc424, navdata, airspace, airport, navaid, airway]
  summary: "Navigation data field definitions (Chapter 5): record fields, coding rules, lengths and character types..."
- id: icao-annex-3 # ICAO Annex 3 (Ed 20) – entry tương tự
```

6. Pipeline ingest & vòng đời section

Thêm một tài liệu mới vào KB là một quy trình có kiểm soát, kết thúc bằng PR review như code. Chi phí LLM chỉ phát sinh một lần tại bước summarize — và từ v0.4, bước này **tự chạy ngay trong kb ingest** qua headless LLM CLI, không cần người mở agent.



Hình 3 — Pipeline ingest: từ PDF gốc đến KB sẵn sàng cho agent truy vấn. Ô cam = con người quyết định.

6.1 Tách section theo bookmark (từ 11/07/2026)

Mặc đị nh `kb ingest` tách PDF theo chính **bookmark/TOC của tài liệu** — các part lấy tên theo outline gốc (chapter, `APPENDIX N` , `ATTACHMENT N` , front-matter...). `--no-bookmarks` quay về tách theo heading pattern (`--chapter-pattern` / `--appendix-pattern` / `--attachment-pattern` , pattern được nhớ từ lần ingest trước). Lưu ý vận hành: **re-ingest là thay thế toàn phần**, không incremental — chạy lại với `--sections 5` thì thư mục doc chỉ còn các file chương 5.

6.2 Vòng đời trạng thái một section



Thay đổi so với v0.2: bỏ kb approve và trạng thái reviewed tự động. v0.2 để CI flip `summarized` → `reviewed` sau merge. Từ v0.9.0, cổng duyệt vận hành duy nhất là **PR publish trên hub** — nội dung không tới được `kb query /MCP/Web` chừng nào PR đó chưa merge, nên field `status: reviewed` trong manifest trở thành **marker tay, tùy chọn**; CLI không tự set nữa. Repo nào muốn dấu "SME đã soát lại từng section" thì đặt tay trước khi merge.

Amendment flow. ICAO Annex / ARINC sửa đổi theo chu kỳ. Khi có bản mới: chạy lại `kb ingest` → `kb diff` tự so với bản cũ → SME chỉ review phần thay đổi → `kb doctor` quét mọi requirement đang cite các section bị ảnh hưởng và cảnh báo cho BA.

7. Retrieval: hybrid search index

Khi BA viết requirement có tag (`#arinc424` `#airspace`) hoặc nêu đích danh tài liệu, resolver tìm đúng section theo thứ tự **từ rẻ đến đắt** — đa số truy vấn dừng ở bước 1–2 với chi phí token gần bằng 0. Điểm mới lớn nhất của v0.10.0: hai bước sau chạy trên **chỉ mục bền vững** (SQLite `search.db` đặt tại `.kb-work/` của bản clone hub) thay vì quét file mỗi lần.

Bước	Cơ chế	Cách hoạt động	Chi phí
1	Exact tag / doc-name match	Tag khớp ở L0/L1; nêu đích danh thì kích hoạt thẳng manifest của nó. Tag cũng dùng để pre-filter phạm vi cho các bước sau.	~0 token, deterministic
2	Keyword — SQLite FTS5	Full-text index trên title + summary + phần đầu L2 của mọi section đã publish; BM25 ranking trong SQLite.	~0 token, chạy bằng code
3	Semantic — vec0 KNN (tùy chọn)	sqlite-vec + fastembed (<code>bge-small-en-v1.5</code> , 384 chiều, chạy local). KNN top-k trên cùng chỉ mục.	Rẻ, chạy local
4	Hợp nhất — RRF	Reciprocal Rank Fusion (k=60, top-50 mỗi leg) trộn hai bảng xếp hạng; thiếu leg semantic thì kết quả là thuần keyword — không lỗi.	~0

Kết quả trả về luôn là **danh sách section đã xếp hạng kèm citation** (`repo-id:doc-id §section (revision)`), và server cắt nội dung theo token budget do client khai báo. Agent không bao giờ nhận về "cả chương" nếu chỉ hỏi một field.

Chỉ mục làm tươi tăng dần, embedding là tùy chọn. `search.db` đồng bộ theo **content-hash** từng section — chỉ re-index/re-embed phần đổi nội dung, mỗi lần query tự sync trước khi tìm. Semantic leg cần extra `pip install "center-kb[embed]"` (~100MB, không kéo PyTorch); thiếu dependency → hệ thống tự chạy keyword-only, và flag `--semantic` giờ chỉ còn tác dụng **cảnh báo rõ** khi embeddings không khả dụng (hybrid vốn đã tự chạy cả hai leg khi có thể).

Ràng buộc đã biết (Windows). SQLite file-lock trên Windows chặt hơn Linux — ghi chỉ mục đồng thời từ nhiều tiến trình trên cùng bản clone hub có thể phải chờ (`busy_timeout` 5s). Với mô hình dùng chung khuyến nghị (một dịch vụ HTTP trên hub), điều này không phát sinh.

8. Federation hub-first, remote MCP & Web UI (Phase 3 —

đã triển khai, kiến trúc lại 13/07/2026)

Mô hình mở rộng cho nhiều repo: mỗi repo giữ `.kb/` riêng làm **bản soạn thảo**; một **kb-hub** trung tâm chứa `federation/` — nơi **duy nhất** mọi máy truy vấn đọc. Đây là thay đổi kiến trúc lớn nhất so với v0.2: trước đây repo con chỉ đẩy chỉ mục L0+L1 lên hub và đọc sâu phải "về đúng repo"; nay `kb publish` mirror **đủ L0→L3**, nên mọi truy vấn cross-repo trả về nội dung đầy đủ, không còn entry `[remote]` bị cắt.



8.2 Remote HTTP MCP + Web UI — một tiến trình

```
CENTER_KB_HTTP_TOKEN=<secret> python -m center_kb.mcp --hub <hub> --transport http
# → agent : http://<host>:8321/mcp      (Bearer token)
# → REST  : http://<host>:8321/api/...  (Bearer token hoặc cookie)
# → người : http://<host>:8321/ui      (đăng nhập bằng token; lưu cookie)
```

Hub là **bắt buộc** (thứ tự resolve: `--hub` → `CENTER_KB_HUB` → `.kb/config.yaml`) — server từ chối khởi động nếu thiếu. Web UI render search + đọc tài liệu, tag là chip bấm được và có thể browse bằng tag không cần keyword. Triển khai chuẩn bằng Docker: `kb docker-setup` sinh `.env` + token, `docker compose up -d`; image phát hành sẵn trên GHCR.

Lưu ý bản quyền. ARINC 424 và các ICAO Annex là tài liệu có bản quyền. Hub giữ mirror đầy đủ (L3 gồm trích xuất nguyên văn theo section) nên **hub bắt buộc là repo private** và HTTP MCP chỉ chạy trong mạng nội bộ/VPN, từ chối khởi động nếu thiếu token. Xem thêm §11.3.

9. Interface: CLI & MCP

9.1 CLI — 15 lệnh

Lệnh	Chức năng	Có từ
<code>kb init [--kind hub child]</code>	Khởi tạo repo KB theo vai trò; scaffold slash commands + MCP wiring cho Claude Code / Copilot / Cursor; re-run refresh scaffold, giữ dữ liệu.	1 · role-aware v0.9
<code>kb docker-setup</code>	(Hub) tạo <code>.env</code> + sinh <code>CENTER_KB_HTTP_TOKEN</code> cho dịch vụ Docker.	v0.9
<code>kb ingest <pdf> --id --tags</code>	Parse + chunk theo bookmark + auto-summarize (tắt bằng <code>--no-summarize</code> / <code>--llm none</code>).	1 · auto v0.4
<code>kb summarize [doc]</code>	Chạy lại/ retry summarize qua headless LLM CLI (claude → copilot), song song <code>--max-workers</code> .	v0.4
<code>kb status</code>	Xem còn bao nhiêu section <code>pending</code> .	1
<code>kb build [--allow-pending]</code>	Validate schema, khớp bảng L2↔L3, cập nhật index, sinh stats.	1
<code>kb query "... " --tags --budget</code>	Truy vấn hub federation, trả section + citation trong token budget; <code>--semantic</code> cảnh báo khi thiếu embeddings.	1·3
<code>kb get <doc> §<sec> --level</code>	Lấy đúng một section ở tầng chỉ định (L2/L3); <code>--repo</code> khi doc-id trùng giữa các repo.	1
<code>kb stats</code>	Token size từng tầng, từng tài liệu — theo dõi chi phí.	1
<code>kb context new --refs "... "</code>	Sinh block <code>kb-context</code> pin commit hub hiện tại — dán vào Jira.	2
<code>kb resolve <file -></code>	Đọc block, trả section đúng bản đã pin + freshness ok/stale/broken.	2
<code>kb diff <doc> --against <rev></code>	So section added/removed/changed giữa hai bản (amendment review).	2
<code>kb doctor [--context]</code>	Kiểm tra sức khỏe: ref stale, index lệch hub, citation gãy, kind chưa ghi.	2·3
<code>kb publish [--pr --direct]</code>	Mirror đủ L0→L3 của repo lên <code>federation/<repo-id>/</code> , mở PR trên hub.	3 · full-mirror v0.9
<code>kb reindex</code>	Rebuild <code>federation/index.yaml</code> khi drift (sửa chữa).	v0.9

So với v0.2: thêm `docker-setup`, `summarize`, `reindex`; bỏ `approve`. Slash commands scaffold sẵn: `/kb-ingest`, `/kb-summarize`, `/kb-publish`, và trên hub `/kb-docker-setup`.

9.2 MCP tools — 4 tool

Tool	Chức năng
<code>kb_search(query, tags?, budget?)</code>	Routing hybrid, trả mọi section liên quan (không chỉ best match) + nội dung L2 trong budget + citation; gắt cờ khi hai kết quả đầu sát điểm — để agent đưa BA xác nhận thay vì tự chọn.
<code>kb_get_section(doc, section, level)</code>	Lấy chính xác một section (dùng khi resolve citation từ ticket).
<code>kb_context_new(refs, tags?)</code>	Pin block <code>kb-context</code> tại commit hub hiện tại từ các ref đã được BA xác nhận — BA làm việc hoàn toàn trong chat, không cần terminal.
<code>kb_resolve(kb_context)</code>	Nhận block <code>kb-context</code> (hoặc cả ticket chứa nó), trả section được cite — đúng version đã pin, kèm freshness.

Phase 4–5 sẽ bổ sung các tool/skill mới (§13–14) nhưng **không đổi** bốn tool lõi này — nguyên tắc "đổi transport / thêm client không đổi engine". So với v0.2 (3 tool): `kb_context_new` đã được đưa lên MCP kèm flow xác nhận citation (12/07/2026) — vốn là một phần nguyên liệu của Phase 4.

9.3 Khai báo trong repo

```
// .mcp.json – kb init scaffold theo vai trò; ai clone là dùng được
// Hub (stdio, chạy tại chỗ):
{ "mcpServers": { "center-kb": {
  "command": "python", "args": ["-m", "center_kb.mcp", "--hub", "."] } } }
// Child (HTTP, trở dịch vụ dùng chung – token qua biến môi trường):
{ "mcpServers": { "center-kb": {
  "type": "http", "url": "${CENTER_KB_MCP_URL}",
  "headers": { "Authorization": "Bearer ${CENTER_KB_HTTP_TOKEN}" } } } }
```

Cursor nhận thêm `.cursor/mcp.json` cùng nội dung; Copilot dùng prompt files tương ứng.

10. Flow theo từng role

Nguyên tắc phối hợp: **một KB duy nhất (trên hub), và chính requirement là sợi dây kết nối.**

Requirement mang theo "context manifest" — block citation máy-đọc-được — để Dev nhận được chính xác nguồn tri thức BA đã dùng.

10.1 Flow của BA (viết requirement)

- 1 **Soạn requirement** — gắn tag nội dung (`#arinc424` `#airspace`) hoặc nêu đích danh tài liệu.
- 2 **Agent tự query KB** qua `kb_search` — nhận **mọi ứng viên** liên quan (L1/L2) kèm citation, trong token budget; nếu hai ứng viên đầu sát điểm, agent phải đưa BA chọn.
- 3 **Viết requirement dựa trên nguồn** — mọi khẳng định về chuẩn ngành phải bám citation; agent chèn sẵn `doc-id §section`.
- 4 **BA xác nhận section áp dụng** → agent gọi `kb_context_new` sinh block `kb-context` (refs + tags + commit hub) và đính vào Jira ticket.
- 5 **Definition of Ready:** requirement liên quan chuẩn ngành mà thiếu citation thì chưa đạt DoR — convention bắt buộc của team. Phase 4 tự động hoá toàn bộ bước 1–5 này.

10.2 Flow của Dev (implement)

- 1 **Mở ticket bằng Claude Code** — agent đọc block `kb-context` trong ticket.
- 2 `kb_resolve(kb_context)` — nhận chính xác các section BA đã dùng, đúng version đã pin. Không search lại từ đầu, không tam sao thất bản.
- 3 **Kiểm tra freshness** — nếu `kb_doctor` báo section đã đổi so với lúc BA viết, trao đổi lại với BA trước khi code.
- 4 **Implement** — kết hợp KB domain + tri thức code structure của repo (`.kb/` code-knowledge, Phase 5); source code vẫn là ground truth.
- 5 **Đóng góp ngược** — phát hiện KB thiếu/mơ hồ thì mở PR vào KB; tri thức quay về hub. Phase 5 đóng gói toàn bộ flow này.

10.3 Flow của SME / Reviewer

- 1 **Review ingest** — duyệt summary do LLM sinh trước khi merge; sửa trực tiếp trên PR của repo nguồn.
- 2 **Review amendment** — khi tài liệu sửa đổi, chỉ review phần `kb_diff` chỉ ra.
- 3 **Duyệt lên hub** — merge PR publish trên hub là hành động đưa nội dung "live"; hub GitHub bật branch protection (require PR).

10.4 Flow của PM

- 1 **Theo dõi sức khỏe KB** — `kb_doctor` chạy định kỳ trong CI: ref stale, citation gãy, index lệch hub.
- 2 **Theo dõi chi phí** — `kb_stats`; LO phình to bất thường là tín hiệu cần tái cấu trúc.
- 3 **Enforce quy trình** — DoR có citation, SME review, KB đổi qua PR (hai lớp: repo nguồn + hub).

11. Governance, versioning & bản quyền

11.1 Versioning & chống drift

- **Pin version:** block `kb-context` trong ticket ghi **một commit hash duy nhất của hub** tại thời điểm BA viết. Từ lúc viết đến lúc code có thể vài tuần — Dev luôn đọc được đúng bản BA đã dùng. (Block hai-version của v0.2 resolve ra `broken` + gợi ý re-pin.)
- **Thay đổi tài liệu là sự kiện phát hiện được:** `kb doctor` đối chiếu mọi ref đang pin với HEAD của hub; section thay đổi → cảnh báo trên ticket, BA xác nhận lại thay vì lỗi ngầm.
- **Git history = audit trail:** trả lời được "requirement TAL-xxxx viết dựa trên ARINC 424 bản nào" — bằng chính lịch sử commit của hub.

11.2 Quyền truy cập theo role

Role	Cửa vào	Phạm vi nhìn thấy
BA	Cowork + remote MCP / Web UI (không cần clone repo)	Toàn bộ federation trên hub (L0-L2, L3 khi cần verify); không cần tầng code.
Dev	Claude Code + MCP trong repo (studio hub / HTTP child)	Toàn bộ: domain + mapping domain→code của repo mình.
SME	Git PR review + <code>/ui</code>	Nội dung KB trước khi merge (repo nguồn) và trước khi live (PR hub).
CI	studio MCP / CLI	Kiểm tra tự động (<code>doctor</code> , <code>gate</code>), <code>kb publish</code> mirror lên hub.

11.3 Bản quyền tài liệu & đóng gói

Lưu ý pháp lý. ARINC 424 và các ICAO Annex là tài liệu có bản quyền/giấy phép. KB dạng summary + trích dẫn chọn lọc phục vụ nội bộ, đặt trong repo **private**, là hợp lý; tuyệt đối không commit raw full-text ra repo công khai hoặc chia sẻ ngoài phạm vi giấy phép tổ chức đang có. Hub giữ mirror đầy đủ nên **hub cũng phải private**; HTTP MCP/Web chỉ nội bộ + token. **Tri thức code (Phase 5) là tài sản riêng của tổ chức** — không vương ràng buộc bản quyền như tài liệu chuẩn ngành, nhưng vẫn ở repo private.

- **PDF nguồn không bao giờ vào Git:** `sources/` bị gitignore cưỡng bức.
- **Package phát hành không chứa dữ liệu:** wheel/sdist chỉ đóng gói `src/` — gate T2 có bước kiểm tra tự động chặn `.kb/` , `sources/` , `tests/` lọt vào artifact. Engine license Apache-2.0; dữ liệu KB tách khỏi engine.

12. Bảng chứng vận hành & chất lượng phát hành

Phần dưới là kết quả đo trên dữ liệu thật (PoC — không đổi từ v0.2) cộng thêm lớp bằng chứng mới: chất lượng kỹ thuật của engine sau 4 ngày productization.

12.1 Dữ liệu đã nạp

Tài liệu	Section	L1 (token)	L2 (token)	L3 (token)	Ghi chú
arinc-424 (Suppl 22, Ch. 5)	325	28.486	76.126	85.669	navdata, airspace, airport...
icao-annex-3 (Ed 20, Ch. 2)	4	420	1.778	2.722	general provisions
L0 index (toàn kho)	187 token				luôn thường trực trong context

12.2 Mức tiết kiệm thật khi truy vấn

Chỉ số	Giá trị
Nạp cả tài liệu gốc (cách ngây thơ)	~400.000 token / câu hỏi
AERO-KB trả về (1–4 section liên quan)	~600–1.200 token / câu hỏi
Mức tiết kiệm mỗi truy vấn	≥ 99% (vượt mục tiêu ≥ 90%)

12.3 Chất lượng & độ tin cậy (PoC)

- `kb build` **PASS** — không có bảng nào lệch giữa L2 và L3, không còn section bỏ dở.
- `kb query` **trả đúng section, đúng citation** (vd `arinc-424 §5.129 (Supplement 22)`), cắt đúng theo token budget.
- **5 lỗi thực tế đã tìm ra và sửa** khi xử lý PDF thật (header lập bị nhận nhầm heading, nhãn "Source/Content:" bị nhận nhầm section...) — đều đã có test tự động chống tái diễn.
- **Giới hạn đã biết:** ~2,4% section của ARINC Ch.5 (6/~250 mục) chưa trích xuất được do lỗi đọc PDF — cần SME đối chiếu thủ công khi gặp.

12.4 Chất lượng kỹ thuật engine (mới từ v0.3)

Hạng mục	Hiện trạng
Phát hành	<code>center-kb v0.10.0</code> — 14 tag release (v0.2.0 → v0.10.0) trong 4 ngày; PyPI + image <code>ghcr.io/vuonglq01685/center-kb</code> .
Release gate 4 tầng	T1 unit/integration (50+ file test) · T2 packaging (wheel/sdist sạch, version khớp tag) · T3 e2e trên wheel đã cài (journey <code>init→summarize→build→publish→query→resolve→doctor</code>) · T4

regression (store v0.7/v0.8/v0.9, MCP contract, golden output, federation compat). Tag đỏ ở bất kỳ tầng nào → không publish gì.

CI matrix	Linux + Windows , Python 3.11–3.13; <code>scripts/gate.sh</code> chạy đúng bộ CI chạy, ngay trên máy dev.
Windows first-class	UTF-8 cưỡng bức ở endpoint, commit LF mọi OS, xử lý read-only file khi publish, hướng dẫn long-path.
Quy trình thiết kế	17 design spec + 17 implementation plan trong <code>docs/superpowers/</code> — mỗi tính năng lớn đều có spec trước khi code.

Kết luận Phần I. Nền tảng không chỉ đủ vững mà đã **đóng gói phát hành được**: hạ tầng tra cứu rẻ, có traceability, chạy multi-repo, cài một lệnh, có Web UI cho người và cổng release có gate. Phần II thiết kế tầng ứng dụng đứng trên — BA Agent (tiêu thụ tri thức để sinh requirement) và Developer Package (vừa tiêu thụ, vừa sản xuất thêm tri thức code).

13. Phase 4 — BA Agent

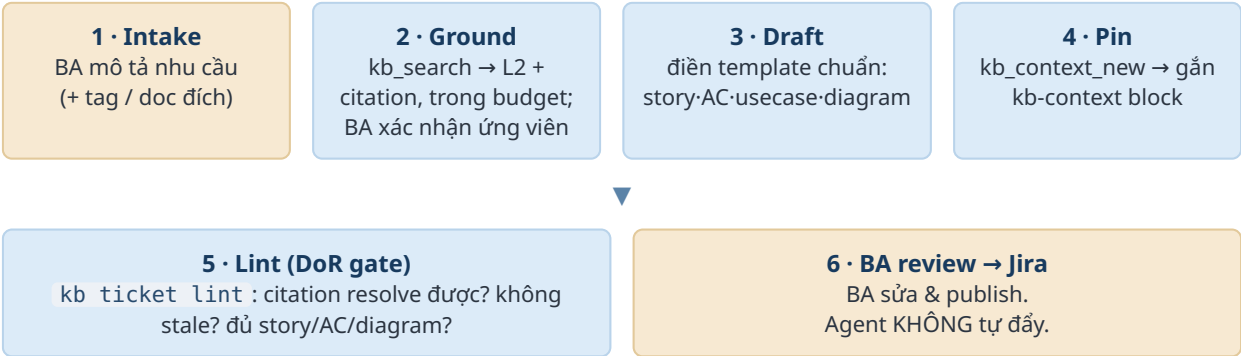
Sinh User Story / ticket chuẩn từ knowledge base, kèm citation, sequence diagram, use case & business flow

13.1 Ý tưởng & điểm tựa

Phase 4 **không xây engine mới** — nó là một **lớp orchestration** (Claude skill + subagent) đứng trên các MCP tool đã có. Toàn bộ nguyên liệu đã sẵn, và so với v0.2 còn sẵn hơn: `kb_search` đã trả **mọi ứng viên + cờ nhập nhằng** để BA xác nhận; `kb_context_new` đã lên MCP nên BA pin citation ngay trong chat; BA không cần clone repo nhờ remote MCP/Web UI. Việc của Phase 4 là **đóng gói các mảnh đó thành một quy trình soạn ticket lặp lại được**, cho ra requirement mà Dev code được ngay.

Đây cũng là bước nối tiếp tự nhiên các skill team đã dùng (`arinc424-popup-builder` map ticket TAL lên DB ARINC, `jira-sprint-monitor` , `mom-formatter`) — BA Agent chuẩn hoá "đầu vào" của chuỗi đó.

13.2 Pipeline sinh ticket



Hình 5 — Pipeline BA Agent: mỗi bước tái dùng một cơ chế đã có ở Phase 1–3.

13.3 Cấu trúc ticket chuẩn (agent sinh ra)

Thành phần	Nội dung & ràng buộc
Title + Summary	Một dòng mô tả nghiệp vụ; ngắn, có đị nh danh.
User Story	"As a <role>, I want <capability>, so that <value>." — bám actor thật.
Background / Business context	Ngữ cảnh nghiệp vụ, mọi khẳng đị nh chuẩn ngành đều cite doc §section .
Acceptance Criteria	Từng AC chạm chuẩn ngành mang citation riêng; đủ điều kiện kiểm thử.
Use cases	Luồng chính + luồng thay thế/ngoại lệ (alternate/exception).
Sequence diagram	Mermaid — actor, system, thứ tự message. Vẽ từ KB + code-knowledge (Phase 5).
Business flow	Mermaid flowchart — điểm quyết đị nh, trạng thái, nhánh.
kb-context block	refs + tags + <code>version</code> (một commit hub) — traceability máy-đọc-được.

Definition of Ready	Checklist tự kiểm: đủ story/AC/diagram, mọi citation resolve được & không stale.
---------------------	--

13.4 Nguyên tắc thiết kế (kế thừa từ KB)

- **Citation bắt buộc, không bị a:** áp đúng style rule của `kb-summarize` — không suy diễn mã hiệu/số liệu; không chắc thì cite và gắn cờ, không tự chế.
- **Xác nhận trước khi pin:** agent phải trình mọi ứng viên `kb_search` trả về và để BA chọn — cơ chế cờ "hai kết quả sát điểm" đã có sẵn trong tool.
- **Diagram accuracy đến từ hai nguồn:** sequence & flow chỉ chính xác khi agent thấy cả tri thức domain (KB) lẫn code structure của repo đích. Đây chính là lý do **Phase 5 làm Phase 4 mạnh hơn hẳn** — agent query được "service nào xử lý restrictive airspace, bảng nào trong DB" để vẽ đúng sequence thật.
- **Diagram-as-code:** Mermaid hợp triết lý docs-as-code, render thẳng trong Jira/GitHub, review được như text.
- **Human-in-the-loop:** agent chỉ tạo bản nháp; BA duyệt và publish. Không auto-push lên Jira.

13.5 Cần xây gì (mức độ)

Hạng mục	Loại	Ghi chú
Skill <code>ba-ticket-author</code>	Mới (prompt/skill)	Template + style rule + orchestration; giống <code>kb-summarize</code> .
Ticket template chuẩn (Jira/Markdown)	Mới (nhẹ)	Story/AC/usecase/diagram/kb-context.
<code>kb ticket lint</code> (DoR gate)	Mới (nhỏ)	Mở rộng <code>kb doctor --context</code> : kiểm cấu trúc + freshness.
<code>kb_search</code> / context new / resolve	Tái dùng	Không đổi engine; flow xác nhận citation đã ship 12/07.

Tổng kết Phase 4: phần lớn là tái dùng; công mới chủ yếu là skill + template + 1 lệnh lint. Đầu tư thấp-trung bình, giá trị cao vì chuẩn hoá "đầu vào" cho toàn chuỗi Dev.

14. Phase 5 — Developer Package

Package cài trong repo con: (a) làm ticket BA giao, (b) liên tục sinh & publish tri thức code

14.1 Hai nửa của package

Package (tạm gọi `aero-kb-dev`, đóng gói dạng plugin) cài vào từng repo con, ứng với hai mục đích người dùng nêu:

Nửa A — Consume (làm ticket)

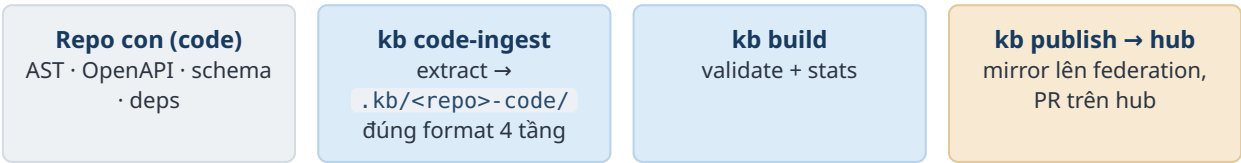
Kết nối MCP của KB (trở hub). Khi Dev mở ticket, agent gọi `kb_resolve` trên block `kb-context` → nhận đúng section BA đã dùng, đúng version pin; `kb_doctor --context` kiểm freshness. **Nửa này đã có sẵn phần lớn từ v0.9** — child-scaffold của `kb_init` đã sinh `.mcp.json` trở HTTP hub (token qua env), slash commands và skill cho Claude Code / Copilot / Cursor. Phần còn thiếu duy nhất: skill "implement ticket này" đọc kb-context và điều phối resolve → freshness → code.

Nửa B — Produce (Codebase-as-Knowledge) — phần kỹ thuật mới của Phase 5

Một **code-aware ingester** sinh `.kb/` docs mô tả chính repo đó, **deterministic (bằng code, không LLM)** — khớp cột "Generated" của mô hình "codebase như một KB": extract bằng code, chạy trong CI mỗi lần merge, **không bao giờ stale**.

Trích xuất	Nguồn & cách làm
Code structure	Danh sách module/service, public API surface, entry point, cây thư mục — từ AST / file tree / convention framework.
API contract	OpenAPI/Swagger nếu có, GraphQL schema, danh sách route — parse file spec hoặc introspect.
Database schema	Bảng, cột, kiểu, quan hệ — từ migration / ORM model / dump schema / <code>.sql</code> . (Team có <code>arinc424.sqlite</code> → extractor SQLite dùng được ngay.)
3rd-party integration	Dependency (package manifest), external service call, endpoint theo env, webhook/queue.
Related repos	Cross-repo dependency: repo này cite doc hub nào, gọi service nào.

14.2 Sinh & publish



Hình 6 — Nửa B: từ code đến tri thức trên hub, chạy tự động trong CI (`kb-code.yml`).

- Lệnh mới `kb code-ingest` phát ra `.kb/<repo>-code/` theo **đúng format 4 tầng** — nên toàn bộ máy query/MCP/federation/hybrid-index đọc được miễn phí, không cần format mới.
- Extract deterministic → tính chất "không bao giờ stale". Có thể thêm một lượt LLM viết *mô tả* L2 (tầng curated), nhưng **sự thật cấu trúc (L3) do máy trích, sinh lại mỗi lần chạy** nên không drift.
- Tự động qua CI: GitHub Action `kb-code.yml` chạy `kb code-ingest` + `kb publish` mỗi lần merge vào main → tri thức code luôn tươi trên hub. Cập nhật theo hub-first: publish là **mirror đủ tầng**

- như mọi repo** (v0.2 viết "chỉ đẩy L0+L1" — đã lỗi thời), và PR trên hub vẫn là cổng duyệt; với nội dung generated có thể cấu hình auto-merge cho PR loại này.
- Namespace riêng (`<repo>-code`) để tri thức generated không trộn với domain doc SME-reviewed và không bao giờ bị sửa tay.

14.3 Hai loại tri thức cùng tồn tại

	Generated (tự sinh)	Curated (con người viết)
Nội dung	API contract, DB schema, module list, dependency graph	Lý do kiến trúc, mapping domain→code, quy ước, edge case đã trả giá
Cách tạo	Extract bằng code, chạy CI mỗi merge	Viết tay, review PR như tài liệu domain
Độ tươi	Không bao giờ stale (sinh lại tự động)	Thay đổi chậm, ít, đáng công duy trì
Nguy cơ	Thấp	Phải có owner; không có vòng feedback sẽ chết dần

Bản thân source code vẫn là ground truth. Agent đọc code trực tiếp khi cần chi tiết; `.kb/<repo>-code/` chỉ giữ phần *tổng hợp* mà đọc code rồi rạc không suy ra nhanh được (bức tranh service, schema tổng, integration map) — đúng để BA Agent (Phase 4) vẽ diagram và để agent định vị nhanh trước khi lặn vào code.

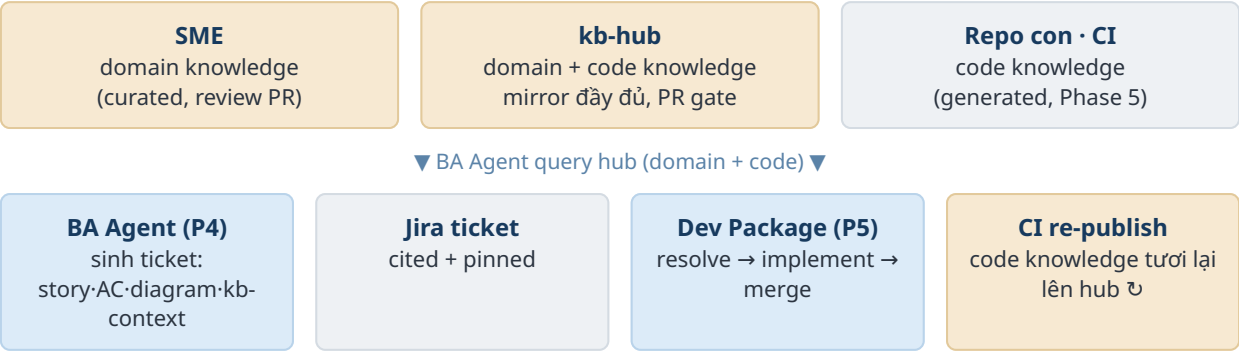
14.4 Cần xây gì (mức độ)

Hạng mục	Loại	Ghi chú
Nửa A — bundle consume (MCP + skill)	Có sẵn ~80%	Child-scaffold v0.9 đã sinh MCP wiring + commands; còn thiếu skill "implement ticket".
Extractor (AST/OpenAPI/SQLite/deps)	Mới	Pluggable, tăng dần; bắt đầu bằng Python + SQLite team đang dùng.
<code>kb code-ingest</code> + <code>kb-code.yml</code>	Mới	Lệnh + CI workflow; tái dùng <code>kb build / publish</code> .

Tổng kết Phase 5: nửa A gần như miễn phí (đã ship phần lớn theo v0.9); nửa B là đầu tư thật nhưng chia nhỏ được theo từng extractor. Bắt đầu với extractor cho stack team đang dùng, mở rộng dần.

15. Vòng khép kín & roadmap tổng hợp

Khi có đủ Phase 4 + 5, hệ sinh thái khép thành một vòng: tri thức domain (SME curate) và tri thức code (tự sinh) cùng chảy qua một hub; handoff BA↔Dev traceable đầu-cuối.



Hình 7 — Vòng khép kín: domain + code knowledge → BA Agent → ticket → Dev → CI publish → hub.

15.1 Roadmap tổng hợp

Phase	Trạng thái	Nội dung	Tiêu chí hoàn thành
1 — PoC	Xong	Package + CLI (ingest/build/query/stats); cấu trúc 4 tầng; ARINC 424 Ch.5 + Annex 3 Ch.2; SME review.	Query đúng section, đúng citation; ≥99% tiết kiệm token.
2 — Workflow	Xong	MCP studio + <code>.mcp.json</code> ; block <code>kb-context</code> ; <code>resolve</code> / <code>diff</code> / <code>doctor</code> ; DoR có citation.	Requirement đi trọn vòng BA→Jira→Dev, resolve được.
3 — Scale	Xong	kb-hub federation + CI publish; remote HTTP MCP; semantic search; auto-summarize.	≥2 repo federation; BA qua remote MCP; doctor bắt drift.
3.5 — Productize (mới, 11–14/07)	Xong v0.10.0	Hub-first single source (bỏ <code>approve</code>); Web UI + REST; hybrid FTS5+vec0 index; role-aware init (hub/child, 3 IDE); Docker/PyPI; release gate T1–T4; Windows.	Gate xanh trên Linux+Windows; cài 1 lệnh; hub là nguồn đọc duy nhất.
4 — BA Agent	Thiết kế	Skill sinh ticket chuẩn từ KB: story, AC, use case, sequence + business flow, citation, kb-context.	Ticket đủ thành phần, mọi citation resolve được, DoR pass tự động.
5 — Dev Package	Thiết kế nửa A ~80% có sẵn	Bundle consume (làm ticket) + code-ingest (structure/DB/integration/related-repo) publish qua CI.	Code knowledge tự sinh, luôn tươi trên hub; Dev làm ticket in-context.

16. Rủi ro & giảm thiểu

Rủi ro	Mức độ	Giảm thiểu
Summary do LLM sinh sai lệch so với tài liệu gốc	Cao	SME review bắt buộc trước merge; bảng/giá trị số giữ nguyên văn; L3 raw luôn sẵn để verify; auto-summarize vẫn đi qua đủ hai cổng PR.
KB curated chết dần vì không ai cập nhật	Trung bình	Vòng feedback Dev→KB là một phần của DoD; PM theo dõi qua <code>kb doctor</code> ; phần generated tự sinh nên không mang phụ thuộc kỷ luật.
Drift giữa lúc BA viết và lúc Dev code	Trung bình	Pin một commit hub trong <code>kb-context</code> ; <code>kb doctor</code> cảnh báo section thay đổi; block định dạng cũ fail rõ ràng (<code>broken</code> + hướng dẫn re-pin).
Vi phạm giấy phép tài liệu chuẩn ngành	Cao	Repo và hub private (hub giữ mirror đầy đủ); HTTP MCP/Web chỉ nội bộ + token; gate T2 chặn dữ liệu lọt vào package phát hành.
Nội dung "live" không ai duyệt	Trung bình	Hub-first: <code>kb query</code> /MCP/Web không thấy gì trước khi PR publish trên hub được merge; hub GitHub bật branch protection.
Chỉ mục hybrid lệch so với federation	Thấp	<code>search.db</code> tự sync theo content-hash mỗi lần query; schema-version của index → đổi schema là rebuild sạch; xoá file là index tự dựng lại.
Phụ thuộc headless LLM CLI khi summarize	Thấp	Tự dò claude → copilot; không có thì section giữ <code>pending</code> và <code>/kb-summarize</code> trong IDE là fallback; build <code>--allow-pending</code> cho làm dở.
P4: BA Agent vẽ diagram sai vì thiếu tri thức code	Trung bình	Ghép Phase 5 (code-knowledge) làm nguồn cho diagram; agent cite nguồn để BA soi; human review trước publish.
P4: agent "bị a" AC không có trong KB	Cao	Style rule không suy diễn; flow xác nhận ứng viên trước khi pin (đã ship); DoR gate <code>kb ticket lint</code> chặn ticket thiếu/gây citation.
P5: extractor sai/không theo kị p framework	Trung bình	Deterministic + chạy lại mỗi merge nên tự sửa; extractor pluggable, thêm dần; code vẫn là ground truth.
P5: tri thức code lộ ra ngoài	Trung bình	Repo/hub private; truy cập qua token, mạng nội bộ; tách namespace <code><repo>-code</code> để kiểm soát phạm vi publish.
Team không theo convention (tag, citation)	Trung bình	Đưa vào DoR/DoD; agent tự chèn citation nên gánh nặng thao tác ~0; lint tự động trong CI.

Phụ lục A — Ví dụ ticket do BA Agent sinh (Phase 4 output)

```
# TAL-1580 – Hiển thị pop-up Restrictive Airspace

## User Story
As a dispatcher, I want to click a restrictive airspace on the map
to see its detail (designation, type, multiple code, level), so that
I can brief the crew accurately.

## Background      # mọi khẳng định chuẩn ngành đều cite
Restrictive airspace record được định danh bởi Restrictive Airspace
Designation, lấy từ nguồn chính phủ [arinc-424 §5.129]. Loại vùng trời
mã hoá theo bảng phân loại P/R/D/A/C/T/W [arinc-424 §5.128].

## Acceptance Criteria
- [ ] Hiển thị các field P1 theo đúng coding rule của UR record      [arinc-424 §5.129]
- [ ] Loại airspace hiển thị theo bảng phân loại                    [arinc-424 §5.128]
- [ ] Dữ liệu đọc từ bảng restrictive_airspace                      [dashboard-code
$db.restrictive_airspace]

## Use cases
- Chính: click vùng có dữ liệu → pop-up hiện đủ field.
- Ngoại lệ: vùng thiếu designation → hiện "N/A" + cảnh báo, không crash.

## Sequence diagram (Mermaid)
sequenceDiagram
    actor D as Dispatcher
    participant UI as Map UI
    participant API as airspace-service
    participant DB as arinc424.sqlite
    D->>UI: click restrictive airspace
    UI->>API: GET /airspace/{id}
    API->>DB: SELECT ... FROM restrictive_airspace
    DB-->>API: row (designation, type, level)
    API-->>UI: airspace detail JSON
    UI-->>D: render pop-up

## Business flow (Mermaid)
flowchart TD
    A[click airspace] --> B{có designation?}
    B -- có --> C[render đủ field]
    B -- không --> D[hiện N/A + cảnh báo]

kb-context:
    version: 7c9e2f41      # MỘT commit duy nhất của kb-hub lúc pin (hub-first v0.9)
    refs:
        - arinc-424 §5.129      # Restrictive Airspace Designation
        - arinc-424 §5.128      # Airspace type codes
        - dashboard-code $db.restrictive_airspace # schema (Phase 5)
    tags: [arinc424, airspace, talora]
```

Lưu ý: sequence & flow bên trên chính xác được là nhờ agent thấy cả tài liệu ARINC (domain) lẫn schema DB & service (code-knowledge Phase 5) — minh hoạ vì sao P4 và P5 bổ trợ nhau. Block `kb-context` theo định dạng một-commit của kiến trúc hub-first; doc-id trùng giữa các repo thì ghi `repo-id:doc-id §section`.

Phụ lục B — Ví dụ code-knowledge doc (Phase 5 output — tự sinh)

Trích `.kb/dashboard-code/_manifest.yaml` (L1) do `kb code-ingest` sinh — cùng format 4 tầng với tài liệu domain, nên query/MCP/hybrid-index đọc được không cần code mới:

```
# .kb/dashboard-code/_manifest.yaml # namespace <repo>-code, generated, không sửa tay
id: dashboard-code
title: "Talora Dashboard – Code Knowledge (generated)"
generated: true
source_commit: "9d1c4e2"
sections:
  - id: "svc.airspace-service"
    title: "airspace-service (REST)"
    summary: "GET /airspace/{id}; đọc restrictive_airspace, controlled_airspace."
    kind: service
    tokens: {l2: 210, l3: 640}
  - id: "db.restrictive_airspace"
    title: "table restrictive_airspace"
    summary: "PK id; designation TEXT, type CHAR(1), multiple_code, min/max level."
    kind: db-schema
    tokens: {l2: 160, l3: 520}
  - id: "int.arinc-sqlite"
    title: "integration: arinc424.sqlite"
    summary: "Local SQLite; nguồn dữ liệu navdata; đọc-only từ dashboard."
    kind: integration
    tokens: {l2: 90, l3: 240}
  - id: "rel.kb-hub"
    title: "related: kb-hub (arinc-424)"
    summary: "Cite arinc-424 §5.128/§5.129 cho airspace pop-up."
    kind: related-repo
    tokens: {l2: 70, l3: 130}
```

AERO-KB Architecture Specification v0.3 — đồng bộ hiện trạng Phase 1–3 (engine CENTER-KB v0.10.0, kiến trúc hub-first) + thiết kế Phase 4–5. Soạn ngày 14/07/2026, thay thế v0.2 (10/07/2026). Tài liệu nội bộ, không phân phối ra ngoài tổ chức.